

Projeto ISP-RAG — Enterprise RAG sobre a Previdência Pública Brasileira

Especificação de projeto — Ledger + Memory + Brain sobre dados públicos do ISP-RPPS ·
Setembro de 2026

1. OVERVIEW

1.1 O Que Vamos Construir

Um **Knowledge Hub sobre a previdência pública brasileira**: um sistema RAG multi-fonte, containerizado, que responde perguntas cruzando três naturezas distintas de conhecimento — o número publicado, o texto normativo que o fundamenta e a rede de relações entre normas, critérios e indicadores.

O sistema ingere os resultados públicos do **Índice de Situação Previdenciária (ISP-RPPS)**, publicados anualmente pelo Ministério da Previdência Social, junto com os relatórios técnicos, notas metodológicas e portarias que os regulamentam. Expõe tudo através de uma API validada por contratos rígidos e consumível por agentes via MCP.

A diferença em relação ao RAG tradicional ("PDF + Vector DB") é arquitetural: a fonte do contexto é ditada pela natureza da pergunta, não pela limitação da ferramenta. "Qual a nota do RPPS de tal município" é consulta relacional. "O que a norma exige" é busca semântica. "O que mudou entre edições e por quê" é travessia de grafo. Um sistema que só tem vetores responde mal a duas dessas três.

1.2 Por Que Este Domínio

A escolha do domínio não é decorativa — ela sustenta três propriedades que projetos de RAG genéricos não têm:

- **Dado real, público e auditável.** Nenhum gerador sintético. Todo número do sistema tem URL de origem, data de coleta e hash de verificação.
- **Série temporal genuína.** São nove edições publicadas do ISP, de 2017 a 2025, para o mesmo conjunto de entes. Perguntas de evolução e comparação entre edições são impossíveis de responder por similaridade vetorial.
- **Metodologia versionada.** O índice mudou ao longo das edições. Isso cria uma classe de pergunta que exige as três camadas simultaneamente — o eixo de demonstração do projeto, descrito em 3.4.

1.3 Stack Tecnológica

- **LlamaIndex**: orquestração — ingestão, roteamento e recuperação.
 - **Pydantic**: contratos de entrada e saída, com fontes citadas obrigatórias.
 - **FastAPI**: camada de serving.
 - **PostgreSQL**: *Ledger* — notas do ISP por ente e por edição, com memória de cálculo.
 - **Qdrant**: *Memory* — corpus normativo e técnico vetorizado.
 - **Neo4j**: *Brain* — grafo de normas, critérios e indicadores.
 - **OpenAI**: embeddings e geração, isolados atrás de uma interface trocável.
 - **Docker Compose**: ambiente local reproduzível.
 - **MCP Server**: expõe o sistema como ferramenta para agentes.
-

2. FONTES DE DADOS PÚBLICAS

2.1 Índice de Situação Previdenciária (ISP-RPPS)

Fonte primária do *Ledger*. O Ministério da Previdência Social publica os resultados do ISP em página própria, com edições de 2017 a 2025. Cada edição disponibiliza:

- Planilhas com o resultado **individualizado por RPPS**, incluindo a memória de cálculo dos componentes;
- Planilha consolidada com o resultado final de todos os entes;
- Relatório técnico final em PDF;
- Notas técnicas, apresentações de metodologia e as portarias que regulamentam o índice.

As planilhas alimentam o *Ledger*; os PDFs alimentam o *Memory*. É essa correspondência — cada documento explicando um número que está no banco — que dá ao roteador algo real para rotear.

2.2 Legislação dos RPPS

Fonte do *Memory* na fase v1. Núcleo do corpus: a **Portaria MTP nº 1.467/2022**, que consolida os parâmetros e diretrizes gerais dos RPPS, mais as Leis nº 9.717/1998 e nº 10.887/2004 e as Emendas Constitucionais 20/1998, 41/2003, 47/2005 e 103/2019.

A relevância é direta: a 1.467 estabelece as obrigações — prazos de DIPR, requisitos de CRP, exigências atuariais — que o ISP mede. Trazê-la para o corpus amplia a classe de perguntas de "qual foi a nota" para "por que a nota é essa e o que a norma exige".

2.3 SICONFI

Fonte opcional do *Ledger* na fase v1.5. Diferente de todas as outras, é uma **API REST viva**, não um arquivo estático — traz dados fiscais (RCL, RREO) que contextualizam a capacidade financeira do

ente.

O valor arquitetural é específico: dá ao sistema a propriedade de "operação viva" sem recorrer a gerador de dados sintéticos. O payload bruto é gravado em coluna `JSONB` na mesma transação que grava o dado normalizado — sem necessidade de um banco de documentos separado.

2.4 Regra de Procedência

Regra inegociável do projeto: todo arquivo entra no sistema por download automatizado a partir da URL pública de origem. Nenhum arquivo entra por cópia manual.

Cada item ingerido registra em um manifesto: URL de origem, data e hora da coleta, tamanho e hash SHA-256. A procedência fica verificável pelo próprio código, e não por declaração no README.

Isso existe por dois motivos. O primeiro é de integridade: o repositório precisa poder ser público, e um único número de origem interna contamina o projeto inteiro. O segundo é de qualidade: uma vez que a memória de cálculo está no banco, o sistema pode **recalcular** o indicador e comparar com o valor publicado. Divergências viram material de análise — é uma checagem de qualidade que quase nenhum projeto de portfólio pensaria em fazer.

3. ARQUITETURA

3.1 Os Três Domínios Cognitivos

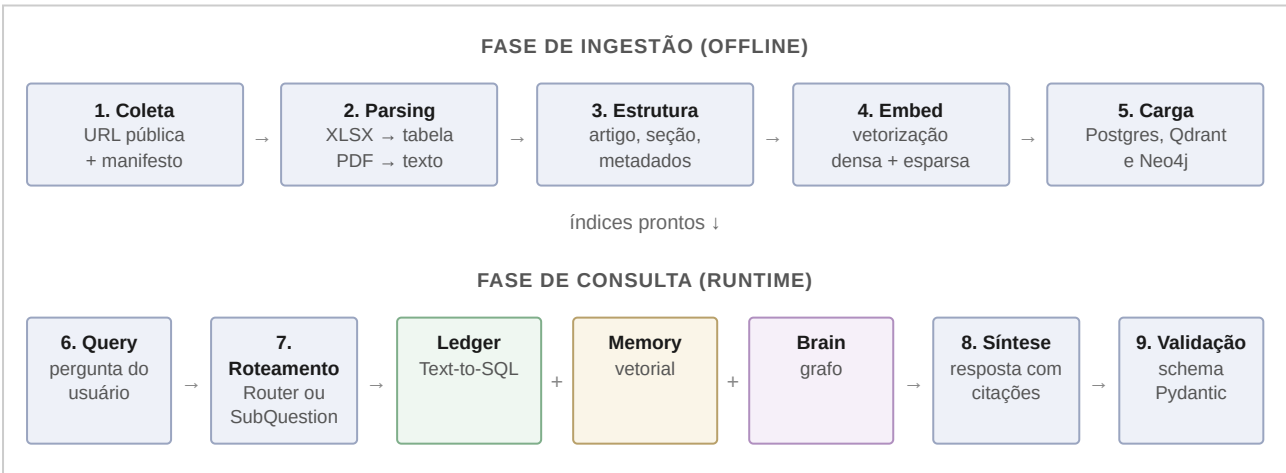
O conhecimento do sistema se divide em três domínios, cada um com tecnologia e padrão de acesso próprios:

- **Ledger (a verdade numérica):** fatos, notas, séries. Não usa vetores. Usa Text-to-SQL sobre PostgreSQL.
- **Memory (o contexto):** normas, relatórios, notas técnicas. Usa busca por similaridade vetorial no Qdrant.
- **Brain (as conexões):** relações, dependências, impacto. Usa travessia de grafo no Neo4j.

3.2 Mapeamento no Domínio RPPS

Camada	Tecnologia	Conteúdo	Fase
Ledger	PostgreSQL + NLSQLTableQueryEngine	Notas do ISP por RPPS × edição, com memória de cálculo	v0
		Edições históricas 2017–2025 (série temporal)	v1
		SICONFI via API (RCL, RREO); payload bruto em JSONB	v1.5
Memory	Qdrant + VectorStoreIndex	Relatórios técnicos, notas técnicas e portarias do ISP	v0
		Portaria MTP 1.467/2022, leis e ECs — chunk por artigo	v1
Brain	Neo4j + PropertyGraphIndex	Ontologia completa modelada; carga inicial: edição → critério → indicador	v2
		Cadeia normativa (revoga/altera) e linhagem indicador ↔ dispositivo	v2+
Procedência	Filesystem + manifesto	PDFs e planilhas originais, com URL, data e hash	v0
Serving	FastAPI + Pydantic	/query com resposta validada e fontes citadas	v0
Avaliação	Gold set + CI	Roteamento, execution match do SQL, fidelidade da resposta	v0
Consumo	MCP Server, deploy, UI em TypeScript	Exposição como ferramenta de agente e interface de consulta	v3

3.3 Pipeline



3.4 A Pergunta de Demonstração

Todo sistema desses precisa de uma pergunta que só ele responde — e ela precisa ser honesta, não montada para o demo funcionar. A do projeto:

"O RPPS de determinado município caiu de B para C entre 2023 e 2025. Foi o desempenho dele que piorou ou a metodologia que mudou? E qual norma alterou isso?"

O *Ledger* devolve as notas das duas edições e o delta por componente. O *Memory* devolve o texto da nota técnica e da portaria aplicável. O *Brain* informa qual edição alterou qual critério e qual norma promoveu a alteração. Nenhuma das três camadas responde sozinha — e é exatamente isso que justifica a arquitetura.

4. DECISÕES DE ARQUITETURA

O projeto adota o padrão de referência de RAG enterprise (*Ledger* + *Memory* + *Brain*, orquestrado por *LlamaIndex*), mas com desvios deliberados. As decisões abaixo são parte do produto: em uma entrevista técnica, explicar o que foi cortado vale mais do que exibir a contagem de containers.

4.1 O Que Se Mantém

PostgreSQL, Qdrant e Neo4j como os três domínios; *LlamaIndex* como orquestrador; *Pydantic* como contrato; *FastAPI* como serving; *MCP Server* como camada de consumo por agentes; *Docker Compose* como ambiente.

4.2 O Que Foi Cortado

Componente	Justificativa do corte
MongoDB	Não existe fonte semi-estruturada que exija um banco de documentos. O único candidato — payload bruto de API — é resolvido por coluna <code>JSONB</code> no PostgreSQL, na mesma transação do dado normalizado, sem sincronizar dois bancos. Um banco a mais precisa se pagar em capacidade de resposta; aqui não se paga.
Data lake S3 (SeaweedFS/MinIO)	Nenhuma consulta toca o armazenamento de origem — ele guarda os arquivos brutos, não responde perguntas. O requisito real é procedência , e isso se resolve melhor com filesystem versionado mais manifesto de hash e URL. Entra na v3 apenas se houver interesse em demonstrar object storage.
Gerador de dados sintéticos	O padrão de referência usa <i>Faker</i> para simular uma operação viva. Aqui há dado real, público e com nove anos de histórico. Substituir dado real por sintético seria perder a maior vantagem do projeto.

4.3 O Que Foi Acrescentado

A **camada de avaliação** (seção 7) e a **regra de procedência** (seção 2.4). Nenhuma das duas consta do padrão de referência, e as duas são o que separa um exercício de arquitetura de um produto que se pode defender.

5. INGESTÃO E INDEXAÇÃO

5.1 Ledger

Modelo relacional com entes e unidades gestoras identificados por CNPJ, resultados por ente e por edição, e os componentes da memória de cálculo em tabela própria — de modo que o recálculo de verificação seja uma consulta, não um script à parte. A chave temporal (edição) é dimensão de primeira classe em todas as tabelas de fato, porque quase toda pergunta interessante é comparativa.

5.2 Memory — Chunking

É a decisão técnica que mais impacta a qualidade final, e o ponto onde a maioria dos projetos de RAG falha.

O erro padrão é cortar o texto a cada N tokens com sobreposição. Em norma jurídica isso destrói o sentido: separa o caput do parágrafo, desliga o inciso do artigo que o rege, e a resposta perde a âncora citável. A estratégia adotada é **um chunk por artigo** — o caput com seus parágrafos, incisos e alíneas — e cada chunk carrega no payload:

- Norma, número, data e órgão emissor;
- Hierarquia completa: Título → Capítulo → Seção → Artigo;
- Situação: vigente, revogado ou alterado, com as datas correspondentes;
- URL da fonte e âncora para o dispositivo;
- O cabeçalho da seção **prefixado no texto indexado** — o que resolve o caso do artigo que diz "o prazo de que trata o caput" sem jamais mencionar o assunto pelo nome.

Artigos longos viram sub-chunks que repetem o caput como contexto. Documentos técnicos (relatórios e notas do ISP) usam a mesma lógica aplicada a seções.

Recuperação híbrida

Busca densa (semântica) combinada com busca esparsa por termo, nativa no Qdrant, fundidas por *Reciprocal Rank Fusion*. A densa resolve "qual o prazo para enviar o demonstrativo"; a esparsa resolve "art. 241" e "DIPR" literalmente. Sobre isso, dois filtros específicos do domínio:

- **Filtro de vigência:** por padrão, recupera apenas dispositivo vigente na data de referência da pergunta.

- **Lookup por citação:** se a pergunta cita um dispositivo explicitamente, a busca vira consulta exata, não vetorial.

5.3 Brain — Ontologia

Princípio de projeto: **schema ambicioso, carga incremental**. A ontologia é modelada completa desde o início — migrar grafo depois é caro — mas povoada por etapas.

Nós: `Norma`, `Dispositivo`, `Edicao`, `Criterio`, `Indicador`, `Ente`. Arestas: `REVOGA`, `ALTERA`, `REGULAMENTA`, `FUNDAMENTA`, `COMPOE`, `CONSOME_CAMPO`.

A carga da v2 cobre o subgrafo obténível a partir dos documentos de metodologia do ISP (edição → critério → indicador). A cadeia normativa completa e a linhagem entre dispositivo e campo de origem do dado entram em seguida. Essa última aresta é, na prática, **rastreabilidade normativa de um sistema de indicadores** — governança de dados expressa em grafo.

6. CAMADA DE CONSULTA

6.1 Roteamento

`RouterQueryEngine` seleciona a engine adequada para perguntas de domínio único. `SubQuestionQueryEngine` decompõe perguntas que cruzam domínios — como a da seção 3.4 — em sub-perguntas roteadas separadamente e sintetizadas ao final. O acerto do roteador é medido, não presumido (seção 7.3).

6.2 Contratos

Pydantic define `QueryRequest` (pergunta, data de referência, filtros opcionais) e `QueryResponse` (resposta sintetizada, sub-perguntas geradas, engines acionadas e — obrigatoriamente — a lista de fontes com identificação do dispositivo ou da tabela de origem). Resposta sem fonte não passa na validação: é erro de contrato, não estilo.

6.3 Geração

Três regras duras no prompt de síntese: responder exclusivamente com base no contexto recuperado; citar norma e dispositivo, ou tabela e edição, em cada afirmação; e declarar ausência de base quando não houver — em vez de preencher a lacuna.

A recusa correta é métrica de sucesso, não falha. Em domínio normativo, um sistema que inventa um prazo é pior que um sistema que diz não saber.

7. AVALIAÇÃO

7.1 Por Que Esta Camada Existe

É o item que separa um sistema que funciona no demo de um sistema que se sabe que funciona. Também é o que mais frequentemente falta em projetos de RAG — inclusive no material de referência que originou este desenho, cuja validação é descrita como "testes cruzados no terminal".

Aqui existem **três coisas distintas para medir**, e cada uma tem métrica própria. Tratá-las como uma só é o erro mais comum na avaliação de sistemas RAG.

7.2 Gold Set

Conjunto de 40 perguntas na v0, expandindo com as fases, escrito manualmente e distribuído em cinco categorias:

- **Fato numérico direto** — resolvida pelo Ledger;
- **Exigência normativa** — resolvida pelo Memory;
- **Comparação entre edições** — exige Ledger e, dependendo da formulação, Brain;
- **Pergunta capciosa** — cita norma revogada ou critério de edição anterior; o sistema deve corrigir a premissa;
- **Sem resposta na base** — o sistema deve recusar.

7.3 Métricas

Alvo	Métrica	Natureza
Roteamento	Acurácia e matriz de confusão sobre perguntas rotuladas por engine esperada	Determinística, barata, roda em segundos
Text-to-SQL	<i>Execution match</i> : o SQL gerado retorna o mesmo resultado do SQL de referência	Determinística. Comparar strings de SQL é métrica errada — dois SQLs diferentes podem estar ambos corretos
Recuperação	Recall@k, MRR e "o dispositivo correto apareceu no top-5?"	Determinística, sem custo de LLM
Geração	Fidelidade ao contexto, precisão da citação, taxa de recusa correta	LLM-as-judge com rubrica explícita; é o custo dominante das rodadas

7.4 Regressão Contínua

A suíte roda em CI a cada mudança relevante, e o README exhibe a **evolução das métricas entre versões** do sistema: chunking ingênuo → chunk por artigo → recuperação híbrida → com filtro de vigência. Um gráfico de melhoria medida comunica competência de engenharia melhor do que qualquer descrição de arquitetura.

8. FASES DE ENTREGA

Construção em fatias verticais: cada fase responde perguntas de ponta a ponta e é entregável por si só. Se o tempo apertar, parar na v1 com tudo medido e documentado é melhor do que uma v3 pela metade sem avaliação.

Fase	Escopo	Capacidade nova
v0	Coleta da edição mais recente do ISP; Postgres com notas e memória de cálculo; Qdrant com relatório técnico e portarias do índice; roteador entre as duas engines; FastAPI <code>/query</code> com validação Pydantic; gold set de 40 perguntas e as métricas de roteamento, SQL e fidelidade	Responde perguntas de fato numérico e de exigência normativa, com fonte citada e desempenho medido
v1	Edições históricas 2017–2025; Portaria MTP 1.467/2022 e legislação correlata no Memory, com chunking por artigo e filtro de vigência	Comparação entre edições; perguntas sobre o que a norma exige, não apenas sobre o resultado
v1.5	SICONFI via API, com payload bruto em <code>JSONB</code>	Contexto fiscal do ente; dado vivo de fonte externa
v2	Neo4j com a ontologia completa; carga do subgrafo edição–critério–indicador; <code>SubQuestionQueryEngine</code> cruzando os três domínios	A pergunta de demonstração da seção 3.4
v3	MCP Server; deploy do Compose; interface de consulta em TypeScript	Consumo por agentes e por usuário final

9. RISCOS

Risco	Mitigação
Contaminação com dado de origem interna	Coleta exclusivamente por URL pública, com manifesto de procedência verificável (seção 2.4). É o risco mais grave: compromete a publicação do repositório.
Qualidade variável dos PDFs	Avaliar extração de HTML estruturado antes de recorrer a parsing de PDF; reservar OCR apenas para documentos digitalizados. Definir isso na primeira semana, porque dimensiona a fase de ingestão.
Heterogeneidade entre edições	As planilhas mudam de formato ao longo dos anos. A v0 trabalha uma única edição justamente para não enfrentar isso antes de o sistema estar de pé.
Explosão de escopo	Fatias verticais com entrega útil em cada fase; componentes cortados (seção 4.2) permanecem cortados salvo necessidade demonstrada por uma pergunta real.
Custo das rodadas de avaliação	Métricas determinísticas rodam sempre; LLM-as-judge roda por amostragem no desenvolvimento e integralmente apenas nos marcos de versão.

10. CUSTOS

A vetorização do corpus é irrisória — poucos milhões de tokens em modelo de embedding pequeno resultam em fração de dólar. O custo relevante é a geração, dominada pelas rodadas de avaliação com LLM-as-judge; em modelo da faixa *mini*, a ordem de grandeza é de US\$ 0,15 por milhão de tokens de entrada e US\$ 0,60 de saída.

Reserva de US\$ 10 a 20 cobre o projeto inteiro com folga, incluindo múltiplas iterações de avaliação. Infraestrutura roda local em Docker; deploy gerenciado, se houver, é o único custo recorrente.

11. COBERTURA DE COMPETÊNCIAS

Mapeamento entre as competências exigidas em posições de engenharia de IA e os entregáveis do projeto:

Competência	Onde é demonstrada
RAG	Arquitetura completa de ingestão, indexação, recuperação híbrida e síntese (seções 3 e 5)
Engenharia de prompt	Prompt de síntese com citação obrigatória e regra de recusa; rubricas de avaliação (6.3 e 7.3)
Avaliação de produtos LLM	Gold set, quatro famílias de métrica e regressão em CI (seção 7)
Padrões agentes	Roteamento por engine, decomposição em sub-perguntas e exposição via MCP Server (6.1 e fase v3)
Python / FastAPI	Toda a camada de serving, com contratos Pydantic
TypeScript	Interface de consulta na fase v3
Engenharia de dados	Coleta versionada, modelagem dimensional, linhagem e procedência auditável
Liderança técnica	Decisões de corte justificadas por capacidade, não por moda tecnológica (seção 4.2)

Sobre fine-tuning

O projeto **não** inclui fine-tuning, e isso é decisão técnica, não omissão. Conhecimento factual que muda a cada edição e a cada alteração normativa se resolve por recuperação, não por peso de modelo: um modelo ajustado sobre a redação de 2024 responderia com confiança sobre uma norma já alterada. O caso em que valeria a pena — treinar um classificador pequeno para roteamento de intenção — está registrado como possibilidade futura, condicionada a que o roteador por LLM se mostre insuficiente na avaliação.

12. REFERÊNCIAS

- Ministério da Previdência Social. *Índice de Situação Previdenciária — Divulgação de Resultados*. Edições 2017–2025. gov.br/previdencia
- Ministério da Previdência Social. *Indicador de Situação Previdenciária — metodologia e notas técnicas*. gov.br/previdencia
- Brasil. *Portaria MTP nº 1.467, de 2 de junho de 2022* — parâmetros e diretrizes gerais dos RPPS.
- Brasil. *Lei nº 9.717/1998; Lei nº 10.887/2004; Emendas Constitucionais nº 20/1998, 41/2003, 47/2005 e 103/2019*.
- Tesouro Nacional. *SICONFI — API de dados fiscais (RREO, RGF)*.
- LlamaIndex. *Documentação: Data Connectors, Query Engines, Property Graph Index*.
- AIDE Brasil. *W01 — Enterprise RAG: LlamaIndex + Pydantic*. Padrão arquitetural de referência.